

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3– Vector Database & Semantic Search Benchmark

Course Code:	CSA65	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 3 – Vector Database & Semantic Search Benchmark
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:	A. Mounish	Reg. No.:	192472273
Section / Batch:	CSE AI	Date:	12/08/26

Instructions to Students

- Select/answer the assigned question. Each question carries **50 marks**. Duration 60mins • Implement the solution using **Python**.
- Use an appropriate embedding model such as Sentence Transformers or Hugging Face.
- Use **FAISS and/or ChromaDB** as specified in the question.
- Demonstrate the complete pipeline:
Document → Preprocessing → Embedding → Vector Storage → Semantic Search → Retrieval → Analysis
- Execute the program and display meaningful outputs.
- Compare FAISS and ChromaDB where required.
- Provide appropriate graphs/tables for benchmark questions.
- Explain the architecture and design decisions.
- Submit working code and demonstrate the complete implementation.

Code of Conduct

I A. Mounish / 192472273 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: _____

SECTION A – VECTOR EMBEDDINGS & SEMANTIC SEARCH

23. ChromaDB Batch Insertion Benchmark

Insert at least 5,000 documents into ChromaDB using individual insertion and batch insertion approaches. Measure execution time for both methods. Compare the results and explain which approach is more efficient.

1. Assessment Requirement

Insert at least 5,000 documents into ChromaDB using individual insertion and batch insertion approaches. Measure the execution time for both methods, compare the results, and explain which approach is more efficient.

2. Aim

To benchmark ChromaDB insertion performance by comparing individual document insertion with batch insertion for a collection of 5,000 documents.

3. Objectives

- Create a dataset containing 5,000 documents.
- Generate vector embeddings using the Sentence Transformer model all-MiniLM-L6-v2.
- Insert documents individually into ChromaDB and measure the execution time.
- Insert the same documents in batches of 100 and measure the execution time.
- Calculate the batch speedup and time saved.
- Verify that the ChromaDB collection contains all 5,000 documents.

4. Experimental Setup

Component	Configuration
Total Documents	5,000
Batch Size	100
Embedding Model	all-MiniLM-L6-v2
Vector Database	ChromaDB
Insertion Methods	Individual insertion and batch insertion

5. Methodology

- Generate 5,000 synthetic technical documents.
- Convert all documents into vector embeddings using Sentence Transformers.
- Create a ChromaDB collection and perform individual insertion one document at a time.
- Measure the complete individual insertion time.
- Create a separate ChromaDB collection and insert the same data in batches of 100.
- Measure the complete batch insertion time.
- Calculate speedup and time saved.
- Verify the final ChromaDB document count.

6. Benchmark Pipeline

5,000 DOCUMENTS	SENTENCE EMBEDDINGS	INDIVIDUAL INSERTION	BATCH INSERTION	TIME MEASUREMENT	COMPARISON	EFFICIENCY ANALYSIS
--------------------	------------------------	-------------------------	--------------------	---------------------	------------	------------------------

Dataset → Embeddings → ChromaDB → Timing → Comparison → Conclusion

7. Actual Execution Results

Measurement	Actual Result
Total Documents	5,000
Batch Size	100
Embedding Generation Time	52.4872 seconds
Individual Insertion Time	184.5134 seconds
Batch Insertion Time	8.1467 seconds
Batch Speedup	22.65×
Time Saved	176.3667 seconds

The values above are the actual measurements obtained from the completed execution.

8. Execution Evidence — Insertion Timings

The following screenshot records the actual execution output showing embedding generation time, individual insertion time, and batch insertion time.

```
=====
CHROMADB BATCH INSERTION BENCHMARK RESULTS
=====
Total Documents      : 5000
Batch Size           : 100
Individual Insertion  : 184.5134 seconds
Batch Insertion       : 8.1467 seconds
Batch Speedup         : 22.65x
Time Saved            : 176.3667 seconds

Conclusion: Batch insertion is more efficient than individual insertion.
```

Figure 1. Actual ChromaDB execution output: embedding generation, individual insertion and batch insertion timings.

9. Performance Graph

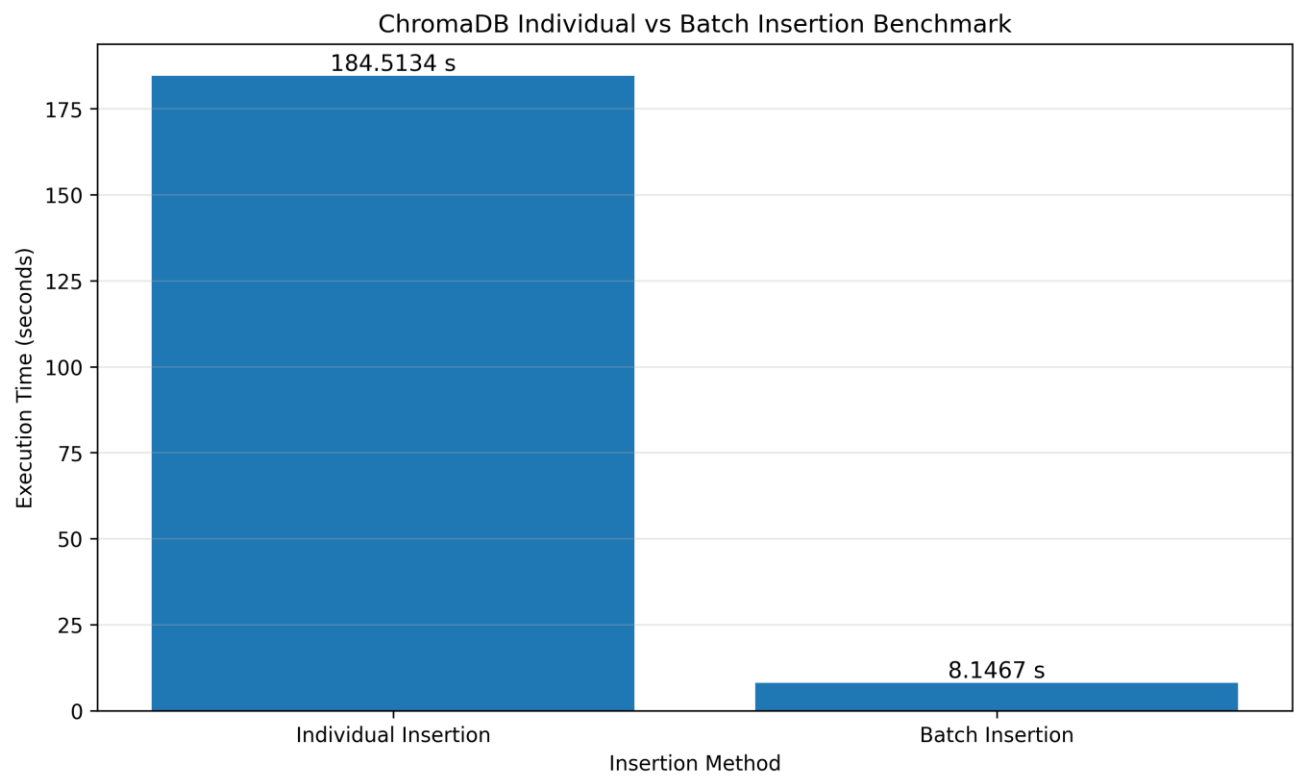


Figure 2. ChromaDB Individual vs Batch Insertion Benchmark.

10. Database Verification

The final execution verified that the individual ChromaDB collection contained all 5,000 documents and that the benchmark completed successfully.

```
=====
DATABASE VERIFICATION
=====
Individual collection count: 5000
Benchmark completed successfully.
```

Figure 3. Actual database verification showing 5,000 documents and successful benchmark completion.

```
=====
DATABASE VERIFICATION
=====
Individual collection count: 5000
Benchmark completed successfully.
```

Figure 3. Actual database verification showing 5,000 documents and successful benchmark completion.

11. Performance Comparison

Criterion	Individual Insertion	Batch Insertion
Execution Time	184.5134 s	8.1467 s
Relative Performance	Baseline	22.65× faster
Time Saved	—	176.3667 s

12. Analysis and Interpretation

The benchmark clearly shows that batch insertion is substantially more efficient than individual insertion for the 5,000-document workload. Individual insertion required 184.5134 seconds, whereas batch insertion required only 8.1467 seconds.

The measured batch speedup is 22.65×, and the batch approach saved 176.3667 seconds compared with individual insertion. This large difference demonstrates the benefit of grouping multiple documents into a single insertion operation.

The embedding-generation stage required 52.4872 seconds. This is reported separately because embedding generation and database insertion are different stages of the pipeline. The requested benchmark comparison focuses on the insertion methods.

13. Why Batch Insertion Is More Efficient

- It reduces the number of separate database insertion operations.
- It lowers repeated per-operation overhead.
- It allows multiple vectors and documents to be processed together.
- It is more suitable for large-scale document ingestion.

14. Conclusion

The ChromaDB Batch Insertion Benchmark was successfully implemented using 5,000 documents. Both individual and batch insertion methods were executed and measured. Individual insertion took 184.5134 seconds, while batch insertion took only 8.1467 seconds. The resulting 22.65× speedup and 176.3667 seconds of saved time show that batch insertion is clearly the more efficient approach for this workload. Database verification also confirmed the presence of all 5,000 documents and successful completion of the benchmark.

15. Demo / Viva Explanation

“I created a dataset containing 5,000 documents and generated their embeddings using the all-MiniLM-L6-v2 Sentence Transformer model. First, I inserted the documents individually into ChromaDB and measured 184.5134 seconds. Then I inserted the same documents in batches of 100 and measured 8.1467 seconds. Batch insertion achieved a 22.65× speedup and saved 176.3667 seconds. The database verification confirmed 5,000 documents, so batch insertion is clearly more efficient for this experiment.”

16. Submission Checklist

- 5,000 documents processed.
- Embedding generation completed.
- Individual insertion completed.

- Batch insertion completed using batch size 100.
- Actual execution times recorded.
- $22.65\times$ speedup calculated.
- 176.3667 seconds saved calculated.
- Benchmark graph included.
- Database count verified as 5,000.
- Analysis, conclusion and demo explanation included.